



Gerenciador de Dependências

NPM

Node Package Manager



O que é um Gerenciador de Dependências?



Analogia: Receita de Bolo

Imagine que você vai fazer um bolo. Em vez de plantar trigo, criar galinhas e extrair açúcar, você vai ao supermercado e compra os ingredientes prontos. Um gerenciador de dependências faz exatamente isso com o seu código: ele busca, instala e organiza os 'ingredientes' (bibliotecas) que você precisa.



Dependência

Um pacote de código reutilizável criado por outra pessoa (ou equipe) que resolve um problema específico, como formatar datas, fazer requisições HTTP ou validar formulários.



Gerenciador

Ferramenta que automatiza o download, instalação, atualização e remoção dessas dependências, além de gerenciar versões e compatibilidades entre elas.



Registro (Registry)

Repositório central online onde os pacotes ficam publicados e disponíveis para qualquer desenvolvedor baixar. No caso do NPM, é o [npmjs.com](https://www.npmjs.com).

Por que usar um Gerenciador de Dependências?

Sem Gerenciador

- Download manual de cada biblioteca
- Copiar arquivos .js para o projeto
- Atualizar manualmente cada versão
- Resolver conflitos de versão na mão
- Sem controle de subdependências
- Difícil replicar o ambiente em outra máquina

Com Gerenciador (NPM)

- Um comando instala tudo automaticamente
- Controle de versões com SemVer
- Atualização segura com um comando
- Resolve subdependências automaticamente
- Lock file garante reprodutibilidade
- Auditoria de segurança integrada

NPM: Node Package Manager

Ferramenta CLI



Interface de linha de comando que vem instalada automaticamente com o Node.js. É o programa que você executa no terminal para gerenciar pacotes.



Registro Online

O maior registro de pacotes do mundo: [npmjs.com](https://www.npmjs.com). Mais de 2 milhões de pacotes disponíveis gratuitamente, desde frameworks como React até utilidades simples.



Ecosistema Node.js

Parte fundamental do ecossistema JavaScript. Suporta projetos frontend (React, Vue, Angular), backend (Express, Fastify) e ferramentas de build (Webpack, Vite).

2M+ pacotes | 17B+ downloads/semana | Usado por 97% dos projetos JS



package.json: O Coração do Projeto

package.json

```
{
  "name": "meu-projeto",
  "version": "1.0.0",
  "description": "Exemplo de projeto",
  "main": "index.js",
  "scripts": {
    "start": "node index.js",
    "dev": "nodemon index.js",
    "test": "jest"
  },
  "dependencies": {
    "express": "^4.18.2",
    "axios": "^1.6.0"
  },
  "devDependencies": {
    "nodemon": "^3.0.1",
    "jest": "^29.7.0"
  }
}
```

name

Nome único do projeto. Deve ser em minúsculas, sem espaços.

version

Versão atual do projeto, seguindo o padrão SemVer.

main

Ponto de entrada do projeto. Arquivo executado ao importar o pacote.

scripts

Comandos automatizados que você define e executa com npm run.

dependencies

Pacotes necessários em produção. Instalados com npm install.

devDependencies

Pacotes usados apenas no desenvolvimento (testes, linters, etc).

node_modules e package-lock.json

node_modules/

- Pasta que contém TODOS os pacotes instalados
- Inclui dependências diretas e suas subdependências
- Pode ter centenas ou milhares de pastas
- Facilmente ocupa centenas de MB
- NUNCA deve ser versionada no Git
- É recriada automaticamente com npm install
- Cada pacote dentro tem seu próprio package.json

package-lock.json

- Arquivo gerado automaticamente pelo NPM
- Registra a versão EXATA de cada pacote instalado
- Garante que todos tenham as mesmas versões
- Inclui hash de integridade para segurança
- DEVE ser versionado no Git
- npm ci usa exclusivamente este arquivo para instalar
- Evita o problema de 'funciona na minha máquina'



Versionamento Semântico (SemVer)

4

MAJOR

.

18

MINOR

.

2

PATCH

MAJOR

Mudança incompatível (Breaking Change)

A API mudou de forma que pode quebrar o código existente. Exemplo: função removida, parâmetro obrigatório adicionado, mudança no retorno.

MINOR

Nova funcionalidade (retrocompatível)

Adicionou algo novo sem quebrar o que já existia. Exemplo: nova função, novo parâmetro opcional, novo evento.

PATCH

Correção de bugs

Corrige um problema sem alterar a API. Exemplo: fix de segurança, correção de cálculo, ajuste de comportamento.

🔍 Símbolos de Versão: ^ (caret) e ~ (til)

Sintaxe	Exemplo	Aceita	Não aceita
<code>^4.18.2</code>	Caret (padrão)	4.18.3, 4.19.0, 4.99.9	5.0.0
<code>~4.18.2</code>	Til (restritivo)	4.18.3, 4.18.9	4.19.0
<code>4.18.2</code>	Exata	somente 4.18.2	qualquer outra
<code>*</code>	Qualquer	todas as versoes	-



Resumo Prático

`^` (caret):

Atualiza MINOR e PATCH, mantendo a MAJOR fixa. É o padrão do npm install.

`~` (til):

Atualiza apenas PATCH, mantendo MAJOR e MINOR fixas. Mais conservador.

Dica:

Use `^` para a maioria dos pacotes. Use versão exata para dependências críticas.

>_ Comandos Essenciais (1/2)

`npm init`

Cria um novo package.json interativamente. Com -y pula as perguntas e usa valores padrão.

`npm install <pacote>`

Instala um pacote e adiciona em dependencies. Alias: npm i. Baixa o pacote e suas subdependências para node_modules.

`npm install`

Sem argumentos, instala TODOS os pacotes listados no package.json. Usa o lock file para garantir versões exatas.

`npm install -D <pacote>`

Instala como devDependency (só para desenvolvimento). Alias: --save-dev. Usado para ferramentas de teste, lint, build.

`npm install -g <pacote>`

Instala o pacote globalmente (disponível em qualquer projeto). Usado para ferramentas CLI como nodemon, create-react-app.

>_ Comandos Essenciais (2/2)

`npm uninstall <pacote>`

Remove um pacote do projeto e do package.json. Alias: `npm rm`, `npm remove`. Também limpa o `node_modules`.

`npm update`

Atualiza pacotes conforme as regras de versão do package.json (respeita `^` e `~`). Atualiza também o lock file.

`npm list`

Mostra a árvore de dependências instaladas. Use `--depth=0` para ver apenas as diretas.

`npm outdated`

Lista pacotes desatualizados mostrando: versão atual, desejada (SemVer) e mais recente disponível.

`npm ci`

Clean Install: apaga `node_modules` e instala EXATAMENTE as versões do lock file. Ideal para CI/CD e ambientes de produção.



dependencies vs devDependencies

dependencies

```
npm install <pacote>
```

Pacotes necessários para a aplicação funcionar em produção. São instalados tanto no desenvolvimento quanto no deploy.

- express - framework web
- react - biblioteca de UI
- axios - requisições HTTP
- mongoose - ODM para MongoDB
- dotenv - variáveis de ambiente
- jsonwebtoken - autenticação JWT

devDependencies

```
npm install -D <pacote>
```

Pacotes usados APENAS durante o desenvolvimento. Não são necessários em produção e não são instalados com `npm ci --production`.

- jest - framework de testes
- nodemon - auto-reload no dev
- eslint - linter de código
- prettier - formatador
- typescript - compilador TS
- webpack - bundler de módulos



Scripts NPM: Automatizando Tarefas

package.json (scripts)

```
"scripts": {  
  "start": "node server.js",  
  "dev": "nodemon server.js",  
  "test": "jest --coverage",  
  "lint": "eslint src/",  
  "build": "webpack --mode production",  
  "format": "prettier --write src/",  
  "db:seed": "node scripts/seed.js",  
  "prestart": "npm run build"  
}
```

Como executar:

`npm start` *Atalho (sem run)*

`npm test` *Atalho (sem run)*

`npm run dev` *Precisa do run*

`npm run lint` *Precisa do run*

`npm run build` *Precisa do run*

`npm run db:seed` *Custom script*



Scripts especiais

start e test

Podem ser executados sem 'run': `npm start`, `npm test`

pre e post hooks

`prestart` executa antes do `start`, `postbuild` executa após o `build`, automaticamente



npx: Executar sem Instalar

O que é o npx?

Ferramenta que vem com o NPM (desde a versão 5.2+) que permite executar pacotes diretamente, sem precisar instalar globalmente. Baixa o pacote temporariamente, executa e descarta. Ideal para comandos que você usa raramente ou para garantir a versão mais recente.

`npx create-react-app meu-app`

Cria um novo projeto React sem instalar o CLI globalmente

`npx create-next-app@latest`

Cria projeto Next.js garantindo a versão mais recente

`npx eslint .`

Executa o ESLint sem instalar globalmente

`npx http-server`

Sobe um servidor HTTP rápido na pasta atual

`npx json-server db.json`

Cria uma API REST fake a partir de um arquivo JSON



Segurança: npm audit

O npm possui auditoria de segurança integrada que verifica suas dependências contra um banco de dados de vulnerabilidades conhecidas. Isso ajuda a identificar e corrigir problemas de segurança antes que afetem seu projeto em produção.

`npm audit`

Escaneia o projeto e lista vulnerabilidades encontradas com níveis de severidade: low, moderate, high, critical.

`npm audit fix`

Tenta corrigir automaticamente atualizando para versões seguras que sejam compatíveis com o SemVer do seu package.json.

`npm audit fix --force`

Força correções mesmo que envolvam breaking changes (MAJOR updates). Use com cuidado e teste tudo depois.

Níveis:

LOW

MODERATE

HIGH

CRITICAL



Boas Práticas com NPM



Nunca versione `node_modules`

Adicione `node_modules/` ao `.gitignore`. A pasta é recriada com `npm install`.



Use `npm ci` em CI/CD

Mais rápido e determinista que `npm install`. Ideal para pipelines automatizados.



Versione o `package-lock.json`

Garante que todos os membros da equipe tenham as mesmas versões instaladas.



Avalie pacotes antes de instalar

Verifique downloads, última atualização, issues abertas e tamanho do pacote.



Execute `npm audit` regularmente

Verifique vulnerabilidades após cada `npm install` e antes de cada deploy.



Documente scripts no README

Explique o que cada script faz para que novos desenvolvedores consigam usar rapidamente.



Arquivos de Configuração

.gitignore

Arquivos que o Git deve ignorar:

Dependências

node_modules/

Variáveis de ambiente

.env

.env.local

Build

dist/

build/

Logs

npm-debug.log*

*.log

.npmrc

Configuração local do NPM:

Salvar versão exata

save-exact=true

Registry customizado

registry=https://registry.npmjs.org/

Engine strict

engine-strict=true

Prefixo de versão

save-prefix=~

☰ Alternativas ao NPM

Yarn

```
yarn add <pacote>
```

Criado pelo Facebook em 2016. Mais rápido que o NPM clássico com cache offline e instalação paralela. Usa yarn.lock. Yarn Berry (v2+) introduziu Plug'n'Play (PnP) que elimina node_modules.

pnpm

```
pnpm add <pacote>
```

Performance-focused. Usa hard links e um store global para evitar duplicação de pacotes. Muito mais eficiente em espaço em disco. Estrutura node_modules não-flat que evita acesso a dependências fantasma.

Bun

```
bun add <pacote>
```

Runtime JavaScript moderno que inclui seu próprio gerenciador de pacotes. Extremamente rápido (escrito em Zig). Compatível com npm, mas com instalação até 30x mais rápida.



Parte Prática

Mão na Massa com NPM

Siga os passos a seguir no seu terminal

Passo 1

Inicializar o Projeto

Terminal

```
# Crie uma pasta para o projeto
```

```
mkdir meu-projeto-npm
```

```
cd meu-projeto-npm
```

```
# Inicialize o projeto (modo interativo)
```

```
npm init
```

```
# OU inicialize com valores padrão
```

```
npm init -y
```



O que acontece?

npm init

Faz perguntas sobre nome, versão, descrição, autor, licença, etc.

npm init -y

Pula todas as perguntas e cria com valores padrão.

Resultado:

Um arquivo package.json é criado na raiz do projeto.

Passo 2

Instalar um Pacote (chalk)

Terminal

```
# Instalar o pacote chalk
npm install chalk@4

# Verificar o que foi instalado
npm list --depth=0

# Ver o package.json atualizado
cat package.json
```

```
index.js

const chalk = require("chalk");

console.log(
  chalk.green("Sucesso!"),
  chalk.red("Erro!"),
  chalk.blue.bold("Info"),
  chalk.yellow("Aviso")
);
```



O que observar?

1. A pasta node_modules/ foi criada com o chalk e suas subdependências
2. package.json agora tem "chalk" em dependencies
3. package-lock.json foi criado/atualizado com versões exatas
4. Usamos chalk@4 (CommonJS). A v5+ usa ESMODULES (import/export)
5. **Execute com: node index.js**

Passo 3

DevDependency + Script Customizado

Terminal

Instalar nodemon como devDependency

```
npm install -D nodemon
```

Verificar package.json

```
cat package.json
```

Executar o script dev

```
npm run dev
```

package.json (editar)

```
"scripts": {  
  "start": "node index.js",  
  "dev": "nodemon index.js",  
  "test": "echo \"Sem testes\"",  
  "hello": "echo Ola NPM!"  
}
```



Passo 4

Remover, Auditar e Explorar

Terminal - exercícios finais

1. Instalar mais um pacote para testar

```
npm install moment
```

2. Verificar dependências instaladas

```
npm list --depth=0
```

3. Remover o pacote moment

```
npm uninstall moment
```

4. Confirmar que foi removido

```
npm list --depth=0
```

```
cat package.json
```

5. Verificar segurança

```
npm audit
```

6. Ver pacotes desatualizados

```
npm outdated
```

7. Explorar o package-lock.json

```
head -30 package-lock.json
```



Resumo

- ✓ NPM é o gerenciador de pacotes padrão do Node.js
- ✓ package.json define o projeto, dependências e scripts
- ✓ SemVer (MAJOR.MINOR.PATCH) controla compatibilidade
- ✓ dependencies vs devDependencies: produção vs desenvolvimento
- ✓ npm audit protege contra vulnerabilidades conhecidas
- ✓ Nunca versiona node_modules, sempre versiona o lock file

Dúvidas?